

# Presentation Agenda

OVERVIEW

**01** Introduction to Software Testing

**02** Key Testing Terminology

**03** 7 Principles of Software Testing

**04** SDLC Overview

**05** STLC Overview

**06** Types of Testing

**07** Levels of Testing

**08** Testing Methods

**09** Manual vs Automated Testing

**10** Defect Lifecycle

**11** Test Case Structure

**12** Introduction to Test Automation

**13** Real-World Testing Example

**14** Best Practices

**15** Common Beginner Mistakes

**16** Summary & Q&A

# 01

## Introduction to Software Testing

*What It Is · Why It Matters · Core Objectives*



# What is Software Testing?

## Definition

Software Testing is the systematic process of evaluating a software application to identify gaps, errors, or missing requirements compared to the actual requirements. It validates that the software behaves as expected under all conditions — both normal and abnormal — before it is delivered to end users.

**Software testing is NOT just about finding bugs — it is about building confidence in quality.**



## Verification

Are we building the product RIGHT? Checking the software against its technical specifications and design documents.



## Validation

Are we building the RIGHT product? Ensuring the final product meets the real-world needs and expectations of the end user.



## Quality Assurance

Proactively preventing defects through process improvement, standards enforcement, and continuous evaluation throughout development.



## Measurement

Quantifying quality through metrics — test coverage, defect density, pass rate — to make data-driven release decisions.

# Why Software Testing is Critical

## The Cost of NOT Testing:

**\$2.84T**

Cost of poor software quality in the USA annually  
(CISQ 2022)

**100×**

Cost to fix a bug in production vs during  
requirements phase

**40%**

Of software projects fail due to quality issues, not  
technical ones

## Core Objectives of Software Testing:

**01**

### Quality Assurance

Ensure the product consistently meets defined quality standards and business requirements before each release. QA is process-oriented — it focuses on preventing defects from entering the codebase in the first place.

**02**

### Defect Detection

Identify and document defects (deviations from expected behaviour) as early as possible. The earlier a defect is found in the SDLC, the cheaper and faster it is to fix — reducing rework and preventing downstream failures.

**03**

### Risk Reduction

Reduce the business and technical risk of releasing software with undiscovered issues. Risk-based testing prioritises effort on the highest-risk areas of the application to maximise quality within time constraints.

## Case Study: NASA Mars Climate Orbiter — \$327 Million Lost (1999)

The orbiter was lost because one engineering team used metric units (Newton-seconds) while another used imperial units (pound-force-seconds) for navigation calculations. No integration testing validated the unit consistency between the two systems. The spacecraft entered the Martian atmosphere at the wrong angle and disintegrated.

### What Was Missing

Integration testing between the two navigation software components was never performed. A simple data validation test would have caught the unit mismatch before launch.

### The Real Cost

\$327.6 million spacecraft destroyed. Years of scientific research lost. Reputation damage to NASA. The mission could not be recovered — unlike a software bug that can be patched.

### The Testing Lesson

Testing is not optional for critical systems. Integration testing specifically validates that independently built components communicate correctly — exactly what was missing here.

### Modern Relevance

Today: financial systems (wrong decimal point = millions lost), medical devices (wrong dosage calculation = patient harm), banking apps (race conditions = double charges).

# 02

## Key Terminology in Testing

*Bug · Defect · Error · Failure · Test Case · Test Scenario · Test Suite*



# Bug - Defect - Error - Failure — The Differences

SECTION 02 | TERMINOLOGY

*These four terms are often used interchangeably but have PRECISE definitions in professional testing:*

## ERROR

*Human Mistake*

An error is a human action that produces an incorrect result. It occurs in the MIND of a developer, analyst, or tester — typically a misunderstanding of requirements or an incorrect logical decision during coding.

*Example: A developer misunderstands that the discount should apply AFTER tax, not before. They write code based on this incorrect understanding.*

## DEFECT

*Code Flaw*

A defect (also called a bug) is the manifestation of an error IN THE CODE. It is a deviation between the actual implementation and the expected specification — a fault that exists in the software artifact.

*Example: The code in the checkout module calculates discount before tax — this incorrect code in the repository is the defect.*

## FAILURE

*Observable Problem*

A failure is what USERS or TESTERS observe when a defect is triggered during execution. It is the external, visible symptom of a defect — the system behaves differently from what was expected or specified.

*Example: During testing, the final price shown is £10 lower than it should be. This wrong output on screen is the failure the tester observes.*

## BUG

*Informal / Colloquial*

'Bug' is the informal, industry-colloquial term encompassing errors, defects, and failures. In day-to-day communication, 'bug' typically refers to a defect in the code. It carries no additional formal precision beyond that.

*Example: A developer says 'I found a bug in the payment module' — meaning they found a defect in the code causing incorrect behaviour.*



## TEST SCENARIO

*Scenarios are broad. One scenario can produce multiple test cases.*

A test scenario is a high-level description of what needs to be tested, derived from a user story or business requirement. It describes a USE CASE — what the user will do — without specifying the exact step-by-step details.

 *Test Scenario: Verify that a registered user can successfully log in to the application using valid credentials.*



## TEST CASE

*One test scenario typically generates 3-10 test cases (valid, invalid, edge cases).*

A test case is the detailed, step-by-step specification of a single testable condition. It includes exact input data, precise steps to execute, expected results, and pass/fail criteria. It is derived FROM test scenarios.

 *TC-001: Enter 'valid@email.com' + 'Correct@Pass1' → Click Login → Expect: Redirected to /dashboard with user name displayed.*



## TEST SUITE

*Suites can be manual (in Jira/TestRail) or automated (TestNG/JUnit XML suite files in Selenium/Playwright).*

A test suite is a collection of related test cases grouped together for a specific purpose — such as testing a feature, module, or regression scope. It provides a structured way to organise, execute, and report on related tests.

 *'Login Module Regression Suite' contains 25 test cases: 8 positive, 12 negative, 5 boundary value cases.*



# 03

## 7 Principles of Software Testing

*The Foundational Laws That Govern All Testing Practice*



## 01 Testing Shows Presence of Defects

Testing can prove that defects EXIST but can NEVER prove that there are NO defects. Even after extensive testing, there may still be undiscovered issues. Testing reduces the probability of undiscovered defects but cannot guarantee zero defects.

📌 *You test 500 login scenarios and all pass. This does NOT mean login is defect-free — it means none of your test cases triggered a known failure.*

## 02 Exhaustive Testing is Impossible

Testing every possible combination of inputs, paths, and conditions is mathematically impossible for any real-world system. A simple login form with 2 fields has millions of possible combinations. Risk-based testing prioritises coverage strategically.

📌 *A 10-character password field using standard ASCII has  $95^{10} \approx 59$  trillion possible inputs. Testing them all at 1 per second =  $\sim 1.87$  million years.*

## 03 Early Testing Saves Money

Testing activities should begin as early as possible in the SDLC — during requirements analysis, not just execution. Defects found in requirements cost 100-200× less to fix than defects found in production. This is the foundation of 'Shift-Left Testing'.

📌 *A requirement ambiguity caught in sprint planning costs 1 hour to clarify. The same issue found in production causes a 3-week hotfix, customer refunds, and reputational damage.*

## 04 Defect Clustering

In practice, a small number of modules or features contain the majority of defects. Roughly 80% of defects are found in 20% of the code (Pareto Principle). This concentration guides where to focus testing effort most intensively.

📌 *In a banking app, the payment processing and interest calculation modules may contain 75% of all defects, while the settings and profile pages have very few.*

## 05 Pesticide Paradox

Running the same test cases repeatedly eventually stops finding new defects. Like pesticides that stop killing insects once they adapt, tests that are never updated lose their effectiveness. Test suites must be reviewed, updated, and extended continuously.

 *Running the same 200 regression tests for 6 months: existing tests catch the same old defects but miss new ones introduced by new features. Rotate and expand.*

## 06 Testing is Context-Dependent

No single testing approach applies to all software. Type, depth, and intensity must be tailored to the application's nature, risk profile, industry regulations, and stakeholder expectations. Context defines the strategy.

 *FDA-regulated medical device: exhaustive formal testing, 100% code coverage, full compliance documentation. A simple to-do app: light manual testing. Same principle, opposite approach.*

## 07 Absence-of-Errors Fallacy

Fixing all known defects does NOT guarantee success. Software built on wrong requirements or that fails to meet real user needs will fail even if technically defect-free. Testing requirements and UX is as important as testing code.

 *A banking app with zero defects — but UX so confusing users can't complete a transfer. Technically perfect. Practically useless. Requirements testing earlier prevents this.*

**Remember: All 7 principles work TOGETHER. A master tester applies every principle on every project.**

# 04

## Software Development Life Cycle (SDLC)

*Understanding Where Testing Fits in the Bigger Picture*



# SDLC Phases & Where Testing Fits

*The SDLC is the structured process used to design, develop, and deliver software. Testing is NOT a single phase — it runs through the ENTIRE cycle.*

## 1. Requirements Analysis



QA reviews requirements for ambiguity, contradictions, and testability. Test Strategy drafted.

## 2. System Design



Test Plan created. Test environments defined. High-level test cases drafted from design docs.

## 3. Implementation (Coding)



Unit tests written by developers alongside code (TDD). Test data prepared.

## 4. Testing & QA



Formal test execution: Functional, Integration, System, Regression, Performance, UAT.

## 5. Deployment



Smoke testing in production environment. Deployment verification checks executed.

## 6. Maintenance



Regression testing on patches. Bug verification. New feature regression cycles.

*Modern software quality is a **SHARED** responsibility — not 'developers build, testers test'.*



## Developers

- Write unit tests for their own code (TDD/BDD)
- Review test cases with QA to ensure coverage
- Fix defects and document root cause analysis
- Provide QA access to builds via CI/CD pipelines



## QA Testers

- Review requirements before developers start coding
- Prepare test data and environments in parallel with dev
- Raise defects with full reproduction steps and evidence
- Verify fixes and close defect loops promptly

## Key Collaboration Practices:

- 1** Daily standups: Testers share blockers, developers share build status — every morning, 15 minutes.
- 2** Test case reviews: QA shares test cases with Dev BEFORE execution — devs can self-test their features first.
- 3** Defect triage: Joint meeting 3x per week — Dev + QA + BA evaluate severity, priority, and root cause together.
- 4** Definition of Done: A user story is only DONE when it has passing test cases, not just working code.

# 05

## Software Testing Life Cycle (STLC)

*The 6 Phases of Structured Testing — From Analysis to Closure*



## 01 Requirement Analysis

QA reviews functional and non-functional requirements with BAs and stakeholders. Testability is assessed. Ambiguous or missing requirements are escalated immediately.

 *Output: RTM (Requirement Traceability Matrix), List of clarifications*

## 03 Test Case Development

Test Analysts write detailed test cases, test scripts, and automated scripts. Test data is prepared. Peer reviews of test cases are conducted. Cases are linked to requirements in the RTM.

 *Output: Test Cases, Automated Scripts, Test Data, Updated RTM*

## 05 Test Execution

Test cases are executed against the build. Defects are logged with full reproduction details. Daily status reports are issued. Failed tests are retested after fixes. Regression cycles run.

 *Output: Test Execution Report, Defect Reports, Retest Evidence*

## 02 Test Planning

Test Manager creates the Test Plan document: scope, schedule, resource allocation, tools, risk register, entry/exit criteria, and estimation. The test strategy is formalised.

 *Output: Test Plan Document, Test Effort Estimation, Resource Plan*

## 04 Test Environment Setup

DevOps and QA collaborate to set up test environments (dev, QA, staging). Tools are installed and configured: Selenium, Playwright, IntelliJ, Eclipse, Jira, Jenkins. Smoke testing confirms environment readiness.

 *Output: Test Environment, CI/CD Pipeline, Entry Criteria Met Confirmation*

## 06 Test Closure

Testing is formally concluded. Metrics are compiled. Test Summary Report is written. Lessons learned are documented. Exit criteria are verified. QA sign-off is issued for release.

 *Output: Test Summary Report, Defect Closure Report, Lessons Learned, Sign-Off*



# 06

## Types of Testing — Overview

*Functional · Non-Functional · Manual · Automated*



# The Four Dimensions of Software Testing

SECTION 06 | TYPES OF TESTING

Every type of testing falls into one or more of these four broad categories. Understanding how they relate helps you plan the right testing approach for any project.

## Functional Testing

Verifies that the software functions according to its specified requirements. It tests WHAT the system does — does the login work? Does the checkout process complete? Does the search return correct results?

 **Tools:** *Selenium, Playwright, Postman (APIs), Cucumber (BDD)*

- Smoke Testing: Basic sanity check after a new build
- Regression Testing: Ensure new code doesn't break old features
- +more

## Manual Testing

Test cases are executed by a human tester without the use of automation tools. The tester interacts directly with the application, observes behaviour, and compares actual results to expected results. Ideal for exploratory, usability, and complex-scenario testing.

 **Tools:** *Jira (defect tracking), TestRail / Zephyr (test management), Browser DevTools*

- Exploratory Testing: Unscripted investigation
- Usability Testing: Human perception of UX
- +more

## Non-Functional Testing

Verifies HOW the system performs under various conditions — not just if it works, but how well. Covers performance, reliability, usability, security, and scalability. Often overlooked by beginners but critical for production readiness.

 **Tools:** *JMeter, Gatling (Performance), OWASP ZAP (Security), Lighthouse (UX)*

- Performance Testing: Speed under normal load
- Load Testing: Behaviour under peak concurrent users
- +more

## Automated Testing

Test scripts are written in code and executed by automation tools without human intervention. Ideal for repetitive, stable, high-volume tests such as regression suites. Provides fast feedback on code changes in CI/CD pipelines.

 **Tools:** *Selenium (Java/Python), Playwright (TypeScript/JS), IntelliJ, Eclipse, Jenkins*

- Regression Automation: Run 500 tests in 20 minutes
- API Testing: Validate endpoints automatically
- +more

# 07

## Levels of Testing

*Unit · Integration · System · User Acceptance Testing*



# The 4 Levels of Testing — The Testing Pyramid

SECTION 07 | LEVELS

Tests are organised into levels that correspond to stages of software development. Lower levels = faster, cheaper, more numerous. Higher levels = slower, more complex, fewer in number.

## L1 — UNIT TESTING

Who: Written by Developers

Tools: JUnit / TestNG (Java), pytest (Python), NUnit (.NET)

Tests individual functions, methods, or classes in complete isolation from the rest of the system. External dependencies (databases, APIs) are replaced with 'mocks' or 'stubs'.

📌 A unit test verifies that the `calculateDiscount(price, rate)` method returns 90 when given `price=100` and `rate=10`.

## L2 — INTEGRATION TESTING

Who: Testers or Developers

Tools: Selenium (basic), REST-Assured, Postman

Tests how multiple components or modules WORK TOGETHER. Validates data flow between integrated units, API contracts, and database interactions.

📌 Integration test verifies: User registers → DB record created → confirmation email triggered by email service — all 3 components validated together.

## L3 — SYSTEM TESTING

Who: QA Team

Tools: Selenium, Playwright, Postman, JMeter

Tests the COMPLETE, INTEGRATED system against the full specification. Functional, performance, security, and regression tests are all executed at system level.

📌 System test: Complete order placement journey — product search → add to cart → enter delivery → payment → order confirmation → stock reduction in DB.

## L4 — UAT TESTING

Who: Business Users / Clients

Tools: Manual execution, TestRail for tracking

The FINAL level before production release. Business users validate the software meets real-world business requirements. Formal sign-off is obtained. Defects at this stage are expensive.

📌 Business analyst tests: 'Can I process an end-of-month payroll run for 500 employees and generate the correct BACS file and payslips?'

# 08

## Testing Methods

*Black Box · White Box · Grey Box Testing*



# Black Box, White Box & Grey Box Testing

SECTION 08 | TESTING METHODS

Testing methods describe *WHAT KNOWLEDGE* the tester has about the system internals when designing and executing tests:

## ■ BLACK BOX TESTING

💡 You receive a locked box. You can only interact with the input/output slots — you cannot see inside.

The tester has NO knowledge of the internal code, architecture, or implementation. Test cases are designed entirely from requirements, specifications, and expected user behaviour. The tester focuses on inputs and observable outputs.

Who: QA Testers, Business Analysts, End Users

Level: System Testing, UAT

Techniques: Equivalence Partitioning, Boundary Value Analysis, Decision Table, State Transition

🔑 Test login: Enter valid credentials → Expect success. Enter invalid → Expect error message. Do NOT look at the authentication code.

✅ No coding knowledge required    ✅ Tests from user perspective    ✅ Finds requirement gaps  
⚠️ Cannot optimise path coverage    ⚠️ May miss internal edge cases

## ■ WHITE BOX TESTING

💡 The box is transparent glass. You can see every wire, circuit, and mechanism inside.

The tester has FULL knowledge of the internal code, logic, and architecture. Tests are designed to exercise specific code paths, branches, loops, and conditions. Requires programming knowledge.

Who: Developers, Senior Automation Engineers

Level: Unit Testing, Integration Testing

Techniques: Statement Coverage, Branch Coverage, Path Coverage, Condition Coverage

🔑 Write a unit test that ensures BOTH the if-branch AND else-branch of the authentication check are exercised, achieving 100% branch coverage.

✅ Thorough code path coverage    ✅ Finds hidden logic errors    ✅ Optimises test efficiency  
⚠️ Requires coding expertise    ⚠️ May miss requirement gaps

## ■ GREY BOX TESTING

💡 You have the box schematics (architecture diagram) but the box itself is still sealed.

The tester has PARTIAL knowledge of internal structure — typically architecture, database schema, APIs, and data flows — but not the complete source code. Combines black box test design with internal knowledge for more intelligent test cases.

Who: QA Leads, Senior Testers, Integration Testers

Level: Integration Testing, System Testing

Techniques: Matrix Testing, Pattern Testing, Orthogonal Array Testing, Regression Testing

🔑 Tester knows the DB schema and API contract. Designs test to verify: API call → correct DB record created → correct JSON response returned.

✅ More targeted than black box    ✅ Accessible without full code access    ✅ Excellent for integration layers  
⚠️ Partial knowledge can create blind spots

# 09

## Manual vs Automated Testing

*Definitions · Differences · When to Use Each · Selenium & Playwright*



# Manual vs Automated Testing — Full Comparison

|                      |  <b>MANUAL TESTING</b>                 |  <b>AUTOMATED TESTING</b>     |
|----------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Definition</b>    | A human tester manually executes test cases by interacting with the application directly — clicking, typing, observing. | Test scripts written in code are executed by automation tools (Selenium, Playwright) without human intervention. |
| <b>Speed</b>         | Slow — a single tester may execute 50-100 test cases per day depending on complexity.                                   | Fast — 500+ test cases can execute in under 30 minutes. Runs unattended overnight.                               |
| <b>Accuracy</b>      | Prone to human error, especially in repetitive tests. Fatigue reduces accuracy over time.                               | 100% consistent execution. A script runs identically every single time, eliminating human error.                 |
| <b>Upfront Cost</b>  | Low — no tooling investment required. Testers need only access and documentation.                                       | High — requires development time to write scripts, maintain frameworks, and manage infrastructure.               |
| <b>Exploratory</b>   | Excellent — humans can intuitively deviate from scripts, notice unexpected UI behaviour, and 'think outside the box'.   | Cannot explore. Automation only does exactly what it is programmed to do. It cannot reason.                      |
| <b>Best For</b>      | New features (unstable), exploratory testing, UX/usability, complex business logic, short-term projects.                | Stable regression suites, repetitive tests, CI/CD pipelines, cross-browser, data-driven, long-term projects.     |
| <b>ROI Threshold</b> | Immediate. Any test case can be executed manually right away.                                                           | Break-even typically at 3-5 re-runs of the same test. Long-term savings are substantial.                         |



# Automation Tools — Selenium & Playwright

SECTION 09 | MANUAL vs AUTOMATED

## Selenium WebDriver

Since 2004

**Languages:** Java, Python, C#, JavaScript, Ruby, Kotlin

The world's most widely-used open-source browser automation framework. Selenium WebDriver communicates directly with browsers using browser-native APIs (ChromeDriver, GeckoDriver). It is the industry standard for web UI automation.

### Architecture:

Test Code → WebDriver API → Browser Driver (e.g. ChromeDriver) → Chrome Browser

### Key Strengths:

- Massive community and extensive documentation
- Supports all major browsers (Chrome, Firefox, Edge, Safari)
- Integrates with TestNG, JUnit, Cucumber, Maven, Gradle
- Works with Jenkins/GitHub Actions for CI/CD
- Used with Eclipse and IntelliJ IDEA

⚠ Requires third-party test runner (TestNG/JUnit). Slower setup than Playwright. Manual waits needed.

🔧 *Selenium + Java + TestNG + IntelliJ: Write PageObject classes, run suite.xml via Maven, publish Allure report.*

## Playwright

Since 2020  
(Microsoft)

**Languages:** TypeScript, JavaScript, Python, Java, .NET

Microsoft's modern end-to-end testing framework designed for contemporary single-page applications. Uses the DevTools Protocol (WebSocket) — faster and more reliable than Selenium's HTTP-based protocol. Has a built-in test runner.

### Architecture:

Test Script → Playwright Library → Browser DevTools Protocol → Chromium / Firefox / WebKit

### Key Strengths:

- Auto-wait: Automatically waits for elements — no manual waits
- Isolated browser contexts: Tests never interfere with each other
- Built-in test runner (Playwright Test) with HTML reporter
- Codegen: Records user interactions as test code automatically
- Trace Viewer: Full execution trace with screenshots and video

⚠ Smaller community than Selenium (but growing rapidly). Best suited for modern SPA frameworks.

🔧 *Playwright + TypeScript: Write tests in .spec.ts files, run via `npx playwright test`, view HTML report with traces.*

# 10

## Defect Lifecycle

*From Discovery to Closure — Full Process Detail*



# Defect Lifecycle — All Stages

*Every defect follows a formalised lifecycle from the moment it is discovered until it is permanently resolved and verified.*

## NEW

Tester discovers a defect during test execution. They log it in the defect tracking tool (Jira / Azure DevOps / Bugzilla) with: unique ID, title, severity, priority, environment, build version, steps to reproduce, expected result, actual result, screenshots/logs.

→ Next: *Assigned*

## ASSIGNED

Test Lead or Test Manager reviews the defect report for validity and completeness. If valid, it is assigned to the responsible developer. If invalid (not a defect, works as designed), it is rejected with justification.

→ Next: *Open*

## OPEN

The assigned developer acknowledges the defect. They begin root cause analysis and start implementing a fix. Status is set to Open once the developer actively starts work. They may request additional information from the tester.

→ Next: *Fixed*

## FIXED

Developer completes the code fix and marks the defect as Fixed. They deploy the fixed build to the QA environment. A brief technical note describing what was changed is added to the defect ticket for QA reference.

→ Next: *Retested*

## RETESTED

The tester re-executes the ORIGINAL failing test case on the fixed build to verify the defect is resolved. They also run a quick impact check to ensure the fix hasn't introduced regressions in adjacent functionality.

→ Next: *Closed or Reopened*

## CLOSED

The defect is confirmed as fixed. Test Manager approves closure. Defect metrics are updated. This is the happy-path end state.

→ Next: —

## REOPENED

The fix was incomplete, incorrect, or introduced a regression. The tester re-opens the defect with detailed evidence (screenshots, log comparison). It re-enters the ASSIGNED state for the developer to re-address.

→ Next: *Open*

## DEFERRED

The defect is acknowledged but its fix is postponed to a future release. Business decision — typically low priority, low impact, or resource-constrained. Must have documented approval from the Product Owner.

→ Next: *Next Release*

# Severity vs Priority — A Critical Distinction

## SEVERITY

Set by: Set by Tester

Measures the TECHNICAL IMPACT of the defect on the functionality of the system. How badly does this defect affect the system's behaviour?

|          |                                                                                            |
|----------|--------------------------------------------------------------------------------------------|
| Critical | System crash, data loss, complete feature unusable, security breach. No workaround exists. |
| High     | Major feature significantly impaired. Workaround possible but difficult or time-consuming. |
| Medium   | Feature works but with minor limitations. Easy workaround available.                       |
| Low      | Cosmetic issues — typos, minor UI misalignment, aesthetic inconsistencies.                 |

## PRIORITY

Set by: Set by Product Owner / BA

Measures the BUSINESS URGENCY of fixing the defect. How quickly does the business need this fixed relative to other work?

|                |                                                                              |
|----------------|------------------------------------------------------------------------------|
| P1 – Immediate | Must be fixed before any further testing. Release is blocked until resolved. |
| P2 – High      | Must be fixed within the current sprint or release cycle.                    |
| P3 – Medium    | Should be fixed in the next planned release if resources allow.              |
| P4 – Low       | Nice-to-fix but can be deferred indefinitely without business impact.        |

## Severity × Priority Matrix — Understanding Conflicts:

| Combination                  | Example                                                                                                  | Decision                      |
|------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------|
| High Severity, Low Priority  | Application crashes on very rare edge case. Fix it eventually but not blocking release.                  | Accepted with documented risk |
| Low Severity, High Priority  | Company logo is wrong on the homepage (visible to millions). No technical impact but high brand urgency. | Fix before release            |
| High Severity, High Priority | Payment checkout is completely broken. Critical blocker.                                                 | Stop-test. Fix immediately    |
| Low Severity, Low Priority   | Footer copyright year shows 2023 instead of 2024.                                                        | Defer to next sprint          |

# 11

## Basic Test Case Structure

*Components, Template & Worked Example*



# Anatomy of a Well-Written Test Case

## **TC ID**

Unique identifier for tracking and traceability. Format: [Module]-[Number].  
Example: LOGIN-TC-001

## **Module / Feature**

The system component being tested. Enables grouping and filtered reporting. Example: User Authentication Module

## **Test Steps**

Numbered, sequential, exact actions the tester performs. Every step must be atomic and unambiguous. Example: 1. Navigate to /login 2. Enter email 3. Enter password 4. Click 'Sign In'

## **Expected Result**

Precise description of what SHOULD happen after executing the steps. Must be specific, not vague. Example: User is redirected to /dashboard. Username 'QA User' is displayed in the header.

## **Status**

Pass / Fail / Blocked / Skipped / Not Executed. Updated after each execution cycle. Drives reporting and release decisions.

## **Test Case Title**

Concise, action-oriented name. Clearly states what is being tested.  
Example: 'Verify successful login with valid credentials'

## **Preconditions**

Conditions that MUST be true before execution begins. Example: User account exists in DB, application is running on QA environment

## **Test Data**

Exact input values to use during execution. Example: Email: qa.user@test.com | Password: ValidP@ss123 | Browser: Chrome 118

## **Actual Result**

What ACTUALLY happened during execution. Filled in only during/after execution. For passed tests: matches expected. For failed: describe the deviation precisely.

## **Priority / Severity**

Importance of the test case. High priority test cases are executed first in constrained time situations.

# Sample Test Case — Login Feature

The following is a complete, professionally written test case following industry standards:

|                 |                                                                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test Case ID    | LOGIN-TC-001                                                                                                                                                                                                    |
| Title           | Verify that a registered user can log in successfully using valid credentials                                                                                                                                   |
| Module          | User Authentication                                                                                                                                                                                             |
| Priority        | High   Severity: Critical   Type: Functional (Positive)                                                                                                                                                         |
| Preconditions   | 1. Valid user account exists: qa.user@test.com / ValidP@ss123 2. QA environment running Build 2.4.1 3. Chrome 118 browser launched                                                                              |
| Test Steps      | 1. Navigate to https://qa.app.com/login<br>2. In the Email field, enter: qa.user@test.com<br>3. In the Password field, enter: ValidP@ss123<br>4. Click the 'Sign In' button                                     |
| Expected Result | User is redirected to /dashboard. Header shows 'Welcome, QA User'. No error messages displayed. Session cookie created successfully.                                                                            |
| Actual Result   | User successfully redirected to /dashboard. Welcome message and name displayed correctly. Session active.  Matches Expected. |
| Status          |  PASSED   Executed by: T. Nkosi   Date: 2024-11-05   Build: 2.4.1                                                            |

# 12

## Introduction to Test Automation

*What It Is · Why It Matters · Tools & Basic Workflow*





# What is Test Automation & Why Does it Matter?

SECTION 12 | AUTOMATION

Test automation is the use of software tools to execute pre-defined test cases automatically, compare actual outcomes to expected outcomes, and report results — without human intervention during execution.

## **Speed**

A regression suite of 500 test cases that takes 2 human-days to execute manually can run in 25 minutes with Playwright or Selenium. This enables multiple testing cycles per day.

## **Repeatability**

Automated tests execute IDENTICALLY every single run. No variations from tester fatigue, distraction, or interpretation differences — guaranteed consistency.

## **CI/CD Integration**

Tests can be triggered automatically on every code commit via Jenkins, GitHub Actions, or Azure DevOps. Defects are caught within minutes of introduction — not days later.

## **Coverage at Scale**

Run 1000+ tests across Chrome, Firefox, and Safari simultaneously using parallel execution. This level of cross-browser coverage is simply impractical manually.

## **Long-Term ROI**

High upfront investment in writing scripts, but once built, re-running 500 automated tests costs near-zero. ROI typically achieved after 3-5 re-runs of the same suite.

## **Continuous Quality**

Automated tests provide real-time quality feedback on every build, transforming testing from a milestone activity into a continuous quality gate embedded in the development process.

# Test Automation Workflow — From Script to Report

SECTION 12 | AUTOMATION

*A professional test automation workflow in a real project using Selenium (Java + IntelliJ) or Playwright (TypeScript):*

## 01 Identify Automation Candidates

Select stable, high-value, repetitive tests for automation. New features (unstable), exploratory tests, and one-off tests stay manual. Typical automation candidates: login, checkout, search, API validation, critical user journeys.

## 03 Write Page Objects

Page Object Model (POM) separates test logic from UI element locators. Create a LoginPage class with methods: enterEmail(), enterPassword(), clickLogin(). Tests call these methods — they don't reference locators directly.

## 05 Integrate with CI/CD

Configure Jenkins or GitHub Actions to trigger ``mvn test`` (Selenium) or ``npx playwright test`` (Playwright) on every code commit or PR merge. Failing tests block the merge — preventing defects from reaching production.

## 02 Set Up the Framework

IntelliJ or Eclipse: Create Maven/Gradle project. Add Selenium or Playwright dependency to pom.xml or package.json. Organise project structure: src/test/java for tests, src/main/java for page objects (POM pattern).

## 04 Write Test Classes

Test classes import page objects and contain `@Test` methods. Each test method: initialises the browser driver, navigates to the URL, calls page object methods, asserts expected outcomes, closes the browser.

## 06 Review Reports & Maintain

Allure or Playwright HTML reporter generates visual test results. Failed tests show screenshots and stack traces. Maintenance: update locators when UI changes, add tests for new features, remove obsolete tests regularly.

# 13

## Real-World Testing Example

*Testing a Login System from Start to Finish*



# Scenario — Test a Login System

*Project: An e-commerce company is releasing a new web application. You are the QA engineer responsible for testing the Login module. Here is how every testing fundamental you've learned applies:*

## Step 1: Requirement Analysis (STLC Phase 1)

- Read the requirement: 'Users should be able to log in using email + password. After 3 failed attempts, account should be locked for 15 minutes.'
- Apply Principle 3 (Early Testing): Identify ambiguity immediately — 'Does the 15-minute lockout reset after one successful login? Is it per session or per account?'
- Raise clarification with BA before any code is written. This prevents a defect from being coded in.

## Step 2: Test Planning

- Define scope: Login form, session management, account lockout, password reset link.
- Estimation: 24 test cases planned. 2 days for design, 2 days for execution, 1 day UAT.
- Risk: Test environment may be unstable — mitigation: provision backup QA environment.
- Tools: Manual execution in Jira Xray + Playwright automated regression scripts in VS Code.

## Step 3: Test Case Design (Black Box + Equivalence Partitioning)

- Positive tests: Valid email + valid password → dashboard. Valid email + password (case variation) → success.
- Negative tests: Valid email + wrong password → error message. Invalid email format → validation error. Empty fields → 'Required' messages.
- Boundary tests: Exactly 3 failed attempts → next attempt triggers lockout. 2 failed attempts → NOT locked.
- Edge cases: Password with special characters @#\$\$%^&\*. Email with + alias. Session expiry after inactivity.

## Step 4: Execution + Defect Found

- Executed TC LOGIN-TC-018 (Boundary: 3rd failed login triggers lockout) — FAILED.
- Actual Result: Account locked after 4 failed attempts, not 3. Defect raised: DEF-041 | Severity: High | Priority: P1.
- Developer fixes lockout counter logic. Retest on Build 2.4.2 — PASSED. TC marked Closed.
- Automation: Playwright script added for all 24 login tests. Runs in 4 minutes on every CI build.

# 14

## Best Practices in Software Testing

*Habits That Separate Good Testers from Great Ones*



01

## Start Testing Early — Shift Left

Engage with requirements from day one. Don't wait for a build. Review user stories, identify ambiguities, and design test cases BEFORE development begins. Every defect prevented at requirements stage saves hours of rework in execution.

 *Tip: Request to be included in sprint planning and refinement sessions.*

03

## Communicate Defects with Evidence

Every defect report must be 100% reproducible from your documentation alone. Include: exact steps, screenshots, video recordings, console logs, environment details, and build number. Developers waste time on vague reports.

 *Tip: Before submitting a defect, ask 'Could I reproduce this from my own report in 3 months?'*

05

## Keep Tests Maintainable

For automation: use the Page Object Model to keep locators separate from test logic. For manual: keep test cases updated as requirements evolve. Outdated tests are worse than no tests — they create false confidence.

 *Tip: Schedule a monthly test case review. Delete obsolete tests, update changed ones, add new ones for recent features.*

02

## Write Clear, Unambiguous Test Cases

Every test case must be executable by ANY competent tester, not just the one who wrote it. Use specific values, clear steps, and precise expected results. Vague test cases like 'check login works' are not test cases.

 *Tip: Peer-review every test case you write. If a colleague can't execute it without asking questions — rewrite it.*

04

## Prioritise Ruthlessly

You will never have enough time to test everything. Risk-based testing focuses effort on high-severity, high-impact areas first. Always know your critical path — the test cases that MUST pass before any release.

 *Tip: On day 1 of execution, always run smoke tests first. Never execute 500 full tests on a broken build.*

06

## Never Assume — Always Verify

Even if a feature 'looks fine' visually, run the test case. Many defects are invisible on screen (database corruption, incorrect calculations, API payload errors). Testers verify, not assume.

 *Tip: Always check the database and API responses, not just the UI, for critical transactions like payments.*

# 15

## Common Beginner Mistakes

*What NOT to Do — Lessons Learned the Hard Way*



# Common Mistakes Made by Junior Testers

SECTION 15 | COMMON MISTAKES

## ⚠️ 01 Skipping Requirements Analysis

*Symptom: 'I'll just start testing once the build is ready.'*

Without understanding requirements, you cannot know WHAT to test, what correct behaviour looks like, or what constitutes a defect. Test cases written without requirements are guesswork.

✅ *Fix: Always read and clarify requirements BEFORE writing a single test case. Raise every ambiguity in writing — never assume.*

## ⚠️ 03 Ignoring Edge Cases and Boundary Values

*Symptom: Testing only the 'normal' happy path — valid username, valid password, click login.*

Most real-world defects live at the boundaries and edges: empty fields, maximum character limits, special characters, null values, zero quantities, negative numbers. Skipping these leaves critical defects undiscovered.

✅ *Fix: For every input field, always test: empty, minimum, maximum, just-over-maximum, special characters, unexpected data types.*

## ⚠️ 05 Starting Test Execution Without Smoke Testing

*Symptom: Running 300 full regression tests against a build that has a broken login page.*

Executing a full test suite against a fundamentally broken build wastes every tester's time. If the basic functionality (navigation, login, core flow) is broken, 80% of your tests will fail for the same root cause.

✅ *Fix: ALWAYS run a smoke test (5-10 critical path tests) against any new build before beginning formal execution. If smoke fails, reject the build and request a fix.*

## ⚠️ 02 Poor or Incomplete Defect Reporting

*Symptom: 'Login doesn't work. Please fix.' — submitted with no other information.*

A vague defect report forces developers to spend hours attempting to reproduce the issue. Most will be returned as 'Cannot reproduce' — wasting everyone's time and delaying the fix.

✅ *Fix: Always include: exact steps, screenshots, expected result, actual result, environment, browser, build number. Make your defect 100% reproducible.*

## ⚠️ 04 Lack of Communication with Developers

*Symptom: Silently logging 20 defects in Jira without talking to anyone. Then wondering why nothing gets fixed.*

Testing is a team sport. Without regular communication, defects pile up unaddressed, blockers last days instead of hours, and release quality suffers. Dev-QA silos kill project velocity.

✅ *Fix: Daily 15-minute sync with the Dev team. Verbal heads-up for Critical/High defects before logging them formally. Joint defect triage 3× per week.*

## ⚠️ 06 Not Updating Test Cases as Requirements Change

*Symptom: Requirements changed 3 sprints ago but the test cases still reflect the old behaviour.*

Outdated test cases create false results — tests 'pass' against the old requirement but the new requirement is never actually tested. Defects slip through because nobody updated the test cases.

✅ *Fix: Treat test cases as living documents. After every requirement change, update the RTM and all affected test cases within 24 hours of the change being approved.*



# 16

## Summary & Key Takeaways

*Everything You Need to Remember*



# Key Takeaways — Your Testing Foundation

SECTION 16 | SUMMARY

**01**

Software testing is not just finding bugs — it is a disciplined engineering practice that builds confidence, reduces risk, and ensures software meets both technical and business requirements.

**02**

Understand the four terms: Error (human mistake) → Defect (code flaw) → Failure (observable symptom). Bug is the informal term for defect. Precision in terminology = professional credibility.

**03**

The 7 Testing Principles are your professional compass. Never claim software is defect-free. Never test exhaustively. Always test early. Rotate your test cases. Consider context always.

**04**

Testing runs through ALL phases of the SDLC, not just the 'Testing phase'. QA testers review requirements, participate in design, and sign off releases — collaborating with developers throughout.

**05**

The 4 Testing Levels form a pyramid: Unit (fast, cheap, many) → Integration → System → UAT (slow, expensive, few). Balance testing effort across all levels for comprehensive coverage.

**06**

Black Box = test what it does. White Box = test how it works. Grey Box = test with partial internal knowledge. Know which method to apply to which testing level and scenario.

**07**

Automation with Selenium and Playwright is essential for regression and CI/CD — but automation supplements manual testing, never replaces it. Exploratory and usability tests will always need human testers.

**08**

Every defect follows the lifecycle: New → Assigned → Open → Fixed → Retested → Closed. Severity is technical impact. Priority is business urgency. They are NOT the same thing.

**09**

A test case has 9 components. Every single one matters. A test case without a precise expected result is not a test case — it is a series of actions with no quality gate.

**10**

The best testers communicate constantly, document meticulously, start testing early, test boundaries aggressively, and never assume a feature works — they verify it.